

Homework 2

6501 Online Optimization and Learning in Games (Fall 2026)

Deadline: 11:59pm, October 11, 2026
Total points: 45

A L^AT_EX template is available at <https://www.overleaf.com/project/6abfab332987b23926974d11/share#48cdbfb01e6ba2e5abb6333b46ebe88ae3948d13a22dd83f>.

1 Distribution-matching / feature-matching game

This problem considers the learning problem where the learner aims to generate samples that resembles some feature distribution μ^* of the world. The learner is able to get samples from μ^* . The goal is to train a model which can generate samples from a distribution $\hat{\mu}$ that is indistinguishable from the true distribution μ^* . The scheme underlies many image generation and imitation learning problems.

Let $\mathbb{B}^d := \{x \in \mathbb{R}^d : \|x\|_2 \leq 1\}$ be the unit ball in d -dimensional space, and let $\mathcal{X} \subseteq \mathbb{B}^d$ be a set of d -dimensional features. Let μ^* be an unknown target distribution on $\mathcal{X}$, and let $\Delta_{\mathcal{X}}$ denote the set of probability distributions on $\mathcal{X}$. A *generator* chooses a sample $x \in \mathcal{X}$, or more generally a mixed strategy $\mu \in \Delta_{\mathcal{X}}$. A *discriminator* chooses a discriminator $h \in \mathcal{H}$, where $h : \mathcal{X} \rightarrow [-B, B]$ for some $B > 0$. Assume that $\mathcal{H}$ is symmetric: if $h \in \mathcal{H}$, then $-h \in \mathcal{H}$. Define the $\mathcal{H}$ -distance between two distributions in $\Delta_{\mathcal{X}}$ by

$$d_{\mathcal{H}}(\mu, \nu) = \max_{h \in \mathcal{H}} |\mathbb{E}_{X \sim \mu}[h(X)] - \mathbb{E}_{X \sim \nu}[h(X)]|.$$

- (a) **(5 points)** Formulate this task as a zero-sum game in which the generator's loss and the discriminator's reward for $x \in \mathcal{X}$ and $h \in \mathcal{H}$ is

$$f(x, h) = \mathbb{E}_{X \sim \mu^*}[h(X)] - h(x).$$

For a distribution over samples $\mu \in \Delta_{\mathcal{X}}$, denote

$$f(\mu, h) = \mathbb{E}_{x \sim \mu}[f(x, h)] = \mathbb{E}_{X \sim \mu}[h(X)] - \mathbb{E}_{x \sim \mu}[h(x)].$$

Show that, for every $\mu \in \Delta_{\mathcal{X}}$,

$$\max_{h \in \mathcal{H}} f(\mu, h) = d_{\mathcal{H}}(\mu, \mu^*),$$

and that the minimax value of the game satisfies

$$\min_{\mu \in \Delta_{\mathcal{X}}} \max_{h \in \mathcal{H}} f(\mu, h) = 0.$$

- (b) **(5 points)** Consider the repeated game for T rounds. On round t , the generator chooses $\mu_t \in \Delta_{\mathcal{X}}$ and the discriminator chooses a discriminator $h_t \in \mathcal{H}$. Define the regret of the generator and the discriminator respectively as

$$R_G(T) = \sum_{t=1}^T f(\mu_t, h_t) - \min_{\mu \in \Delta_{\mathcal{X}}} \sum_{t=1}^T f(\mu, h_t)$$

and

$$R_D(T) = \max_{h \in \mathcal{H}} \sum_{t=1}^T f(\mu_t, h) - \sum_{t=1}^T f(\mu_t, h_t).$$

Define its average strategy $\bar{\mu}_T$ by

$$\bar{\mu}_T = \frac{1}{T} \sum_{t=1}^T \mu_t.$$

Prove that

$$d_{\mathcal{H}}(\bar{\mu}_T, \mu^*) \leq \frac{R_G(T) + R_D(T)}{T}.$$

Briefly explain what this implies when $R_G(T) + R_D(T) = o(T)$.

In the next two subproblems, we design no-regret algorithms for both the generator and the discriminator under simplification assumptions.

- (c) **(5 points)** Suppose that $\mathcal{X}$ is finite. Give an explicit no-regret algorithm for updating the generator's distribution from μ_t to μ_{t+1} after observing h_t . Recall that μ^* is unknown. We further assume that the generator does not have sampling access to μ^* . Therefore, the update rule should not involve μ^* .
- (d) **(5 points)** Suppose the discriminator class consists of linear predictors

$$\mathcal{H} = \{h_{\theta} : \theta \in \mathbb{R}^d, \|\theta\|_2 \leq B\}, \quad h_{\theta}(x) = \langle \theta, x \rangle.$$

We will run an online linear optimization algorithm to update θ_t and set $h_t = h_{\theta_t}$. This is valid because $f(\mu, h_{\theta})$ is a linear function of θ for any μ . Specifically, let's use the projected gradient ascent algorithm from Problem 3(e) of HW1. On round t , assume that the discriminator can draw a target sample $X_t^* \sim \mu^*$ and receives a generated sample $X_t \sim \mu_t$. Use X_t^* , X_t , and θ_t to construct a gradient estimator g_t such that

$$\mathbb{E}_{X_t \sim \mu_t, X_t^* \sim \mu^*} [g_t \mid \theta_t] = \nabla_{\theta} f(\mu_t, h_{\theta}) \big|_{\theta=\theta_t}.$$

Then describe how θ_{t+1} should depend on X_t^* , X_t , θ_t under projected gradient ascent with learning rate η .

Remark 1. This is a toy modeling of the generative adversarial network (GAN), whose goal is to let the generator generate samples whose distribution matches that of the target distribution μ^* . However, the discriminator class plays an important role in how we measure the closeness between the generator's sample distribution and the target distribution. For example, with the linear discriminator class in (d) above, we have

$$d_{\mathcal{H}}(\mu, \nu) = \max_{\theta: \|\theta\|_2 \leq B} |\langle \theta, \mathbb{E}_{X \sim \mu}[X] - \mathbb{E}_{X \sim \nu}[X] \rangle| = B \cdot \|\mathbb{E}_{X \sim \mu}[X] - \mathbb{E}_{X \sim \nu}[X]\|_2.$$

Therefore, the guarantee in (b) only ensures the generator's *feature mean* $\mathbb{E}_{X \sim \bar{\mu}_T}[X]$ matches that of the target distribution $\mathbb{E}_{X \sim \mu^*}[X]$; it does not ensure that their *full feature distributions* are matched. If we instead let $\mathcal{H}$ contain all bounded 1-Lipschitz functions, then $d_{\mathcal{H}}(\mu, \nu)$ is a Wasserstein distance between μ and ν , which is more discriminative and provides a stronger distribution-matching notion (Arjovsky, Chintala, and Bottou, 2017). If, even more discriminative, $\mathcal{H}$ contains all possible bounded functions, then $d_{\mathcal{H}}(\mu, \nu)$ is a total-variation distance (defined in Problem 4 of HW1) between μ and ν .

Remark 2. In practice, we are unable to enumerate features and maintain a distribution over them like in (c) above. Instead, we train a generator of the form $x_{\phi} : \mathbb{R}^m \rightarrow \mathbb{R}^d$ parameterized by ϕ . With random input $\varepsilon_t \in \mathbb{R}^m$, $X_t = x_{\phi_t}(\varepsilon_t)$ is a random sample in $\mathbb{R}^d$, and thus ϕ_t implicitly defines μ_t . We update the parameter with $\phi_{t+1} \leftarrow \phi_t + \eta \frac{\partial}{\partial \phi} [h_t(x_{\phi}(\varepsilon_t))]$.

2 Approximately solving linear programming via no-regret algorithms

The application studied here is adapted from Section 3.4 of [Arora, Hazan, and Kale \(2012\)](#). Feel free to use the paper to assist answering this question.

A recipe for solving LPs via no-regret learning. Consider a linear program of the following form:

$$\begin{aligned} & \text{maximize} && c^\top x \\ & \text{subject to} && Ax \leq b, \quad (\text{difficult constraints}) \\ & && x \in \mathcal{K}, \quad (\text{easy constraints}). \end{aligned} \tag{LP}$$

The distinction between easy and difficult constraints depends on the problem. The set of easy constraints should be chosen to have the following properties:

- $\mathcal{K}$ is bounded.
- Linear optimization over $\mathcal{K}$ is *easy*: we can compute $\operatorname{argmin}_{x \in \mathcal{K}} g^\top x$ easily for g of a particular form $g = \tilde{A}^\top y$ (defined below).

We follow three steps:

1. **Step 1: Convert optimization to feasibility.** For a target objective value U , consider the feasibility problem

$$\begin{aligned} & \text{find} && x \in \mathcal{K} \\ & \text{such that} && c^\top x \geq U, \\ & && Ax \leq b. \end{aligned} \tag{FS}$$

or certify that no such x exists. This allows a binary search over U . Combine the constraints $Ax \leq b$ and $c^\top x \geq U$ into $\tilde{A}x \leq \tilde{b}$. Assume $\tilde{A} \in \mathbb{R}^{N \times d}$.

2. **Step 2: Convert feasibility to a two-player zero-sum game.** Define $f(x, y) = y^\top (\tilde{A}x - \tilde{b})$ and define

$$v^* = \min_{x \in \mathcal{K}} \max_{y \in \Delta_N} f(x, y). \tag{GAME}$$

The feasibility problem (FS) has a solution if and only if $v^* \leq 0$. In that case, any minimizer of the minimax objective is a feasible solution of (FS).

3. **Step 3: Run no-regret–best-response dynamics.** Approximately solve (GAME) by running the no-regret–best-response dynamics. In particular, let the maximizer player run a no-regret algorithm, and the minimizer player run best response. Note that the best response to y is $x = \operatorname{argmin}_{x \in \mathcal{K}} x^\top (\tilde{A}^\top y)$ and hence we require this to be easy to solve.

Application: maximum flow. Let $G = (V, E)$ be a directed graph where V is the set of vertices and E is the set of edges. Each edge $e \in E$ has capacity $c_e > 0$. Fix source and sink $s, t \in V$, and assume that there is at least one directed s – t path. Let $\mathcal{P}$ be the set of simple directed s – t paths.

For every path $P \in \mathcal{P}$, let $x_P \geq 0$ denote the amount of flow routed along P . We will use the following path-flow formulation of maximum flow:

$$\begin{aligned} & \text{maximize} && \sum_{P \in \mathcal{P}} x_P \\ & \text{subject to} && \sum_{P \in \mathcal{P}: e \in P} x_P \leq c_e, \quad e \in E, \\ & && x_P \geq 0, \quad P \in \mathcal{P}. \end{aligned} \tag{MF-1}$$

- (a) **(5 points)** Let $C = \sum_{e \in E} c_e$. Consider the following modification of (MF-1):

$$\begin{aligned}
& \text{maximize} && \sum_{P \in \mathcal{P}} x_P \\
& \text{subject to} && \sum_{P \in \mathcal{P}: e \in P} x_P \leq c_e, \quad e \in E, \\
& && \sum_{P \in \mathcal{P}} x_P \leq C, \\
& && x_P \geq 0, \quad P \in \mathcal{P}.
\end{aligned} \tag{MF-2}$$

Argue that the additional constraint $\sum_{P \in \mathcal{P}} x_P \leq C$ is redundant, so (MF-2) has the same feasible set and optimal value as (MF-1).

- (b) **(5 points)** Fix a target flow value F with $0 < F \leq C$, and define the easy constraint set

$$\mathcal{K} = \left\{ x \in \mathbb{R}^{\mathcal{P}} : \sum_{P \in \mathcal{P}} x_P \leq C, \quad x_P \geq 0, \quad P \in \mathcal{P} \right\}.$$

The converted feasibility problem is

$$\begin{aligned}
& \text{find} && x \in \mathcal{K} \\
& \text{such that} && \sum_{P \in \mathcal{P}} x_P \geq F, \\
& && \sum_{P \in \mathcal{P}: e \in P} x_P \leq c_e, \quad e \in E.
\end{aligned} \tag{MF-FS}$$

Follow the recipe described previously to convert (MF-FS) into a minimax problem:

$$\min_{x \in \mathcal{K}} \max_{y \in \Delta_N} f(x, y). \tag{MF-GAME}$$

where $N = |E| + 1$. Write down the objective function $f(x, y)$ corresponding to (MF-FS).

- (c) **(5 points)** Assume that we already have a procedure to solve the following best response for any given $y \in \Delta_N$:

$$x^*(y) = \operatorname{argmin}_{x \in \mathcal{K}} f(x, y) \tag{1}$$

Design an algorithm that runs the no-regret-best-response dynamics for T rounds to approximately solve (MF-GAME) using (1) as a subroutine. The algorithm takes in f and produces two outputs:

- $\bar{v} \in \mathbb{R}$: an approximation of the minimax value $\min_{x \in \mathcal{K}} \max_{y \in \Delta_N} f(x, y)$.
- $\bar{x} \in \mathbb{R}^{\mathcal{P}}$: an approximation of the minimax solution $\operatorname{argmin}_{x \in \mathcal{K}} \max_{y \in \Delta_N} f(x, y)$.

Write down the pseudocode.

- (d) **(5 points)** Now address missing piece of the algorithm in (c) — solving $x^*(y) = \operatorname{argmin}_{x \in \mathcal{K}} f(x, y)$. In our problem, does this correspond to another well-known graph problem? Are there existing algorithms to solve it?
- (e) **(5 points)** Finally, we would like to approximate the maximum flow value and constraint satisfaction to an error of ε , i.e., outputting x_P such that

$$\begin{aligned}
& \sum_{P \in \mathcal{P}} x_P \geq F^* - \varepsilon, \\
& \sum_{P \in \mathcal{P}: e \in P} x_P \leq c_e + \varepsilon, \quad e \in E, \\
& x_P \geq 0, \quad P \in \mathcal{P},
\end{aligned}$$

where F^* is the true maximum flow value.

Write down the pseudocode of the binary search procedure that calls the algorithm in (c) as a subroutine. Specify the T in (c) needed to achieve the error of ε .

Remark. The final solution produced in (e) may violate the capacity constraints. The slightly more involved version in [Arora, Hazan, and Kale \(2012\)](#) can avoid this.

3 Survey

Leave any feedback for the course.

References

- Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: A meta-algorithm and applications. *Theory of Computing*, 8(6):121–164, 2012. <https://theoryofcomputing.org/articles/v008a006/v008a006.pdf>.
- Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223, 2017. <https://arxiv.org/pdf/1701.07875>.